Voter2 Transition Guide
(From Voter/RTCM)

BY: WILL BEALS

DRAFT VERSION: 1.3

AUGUST 20, 2026

An Open-Source Hardware and Software Project
www.colcon.org
www.rmham.org

1 INTRODUCTION

Realizing that almost everyone using a Voter2 will have first had at least some if not extensive experience with the Voter/RTCM already, this is a short guide to help you get started with the Voter2 assuming you are already familiar with the Voter/RTCM.

From here on, I will refer to the Voter/RTCM as just the Voter.

It is important to note that the Voter2 is a ground-up project, not borrowing any code from the Voter (this is explained in the Users Guide). I had very little experience with the Voter before embarking on the Voter2 and even during Voter2 development used it very little, mainly to ensure interoperability, never in an actual "real world" environment. As such, I am aware I have almost certainly made some usage assumptions that are not normal operating practice. I look forward to that feedback so I can make it a better project over time.

2 SCOPE

This is not intended to be a replacement for the Hardware Guide or User Guide documents. They are the reference documents for the Voter2. This is intended to be a cheat sheet for folk already familiar with the Voter to make the transition to the Voter2 a bit easier, highlighting the similarities, differences, and enhancements over the Voter. It just notes these differences to save some flailing and know what to look for in the Hardware and Users guides.

3 OTHER DOCUMENTS

Here are the full list of documents for the Voter 2. All are on the Github site, but in different directories depending on their association

- **Voter2 Users Guide:** Primarily centered around the setup and operation of the Voter2. Heavily tied to software releases. This document is in the software release directories and is in sync with the software release in that directory. In theory, this guide will be applicable to all Voter2s (Voter2-2K, Voter2-SA, possibly others)
- **Voter2 Hardware Reference Manual:** Primarily centered around the board hardware including board specifications, block diagrams, reasoning behind chip choices and describing the interfaces in detail. High level software architecture is covered as well. This document is in the hardware release directories and will be synchronized with the hardware revision level. The primary document will be for the Voter2-2K with the Voter2-SA being a "differences" document.
- **Voter2 Transition Guide:** (this document) This is a short document intended for folk already familiar with the Voter/RTCM and looking for a quick primer on how to

get up and running with the Voter2. It highlights the similarities and differences from the Voter/RTCM. It is high level on purpose, not duplicating too much information in the other manuals. It is stored in “Other Documents” on Github.

- **Voter2 Quick Start Guide:** Another short document covering the core commands needed to bring up a Voter2 from factory defaults. It is stored in “Other Documents” on Github.

4 INTEROPERABILITY

4.1 PROTOCOL

By requirement, the Voter2 is fully compatible with the Voter protocol as implemented by the ASL1 and ASL3 platforms. The Voter2 must operate with these platforms without any modifications to them. As with any non-trivial protocol there are nuances and boundary cases I've run in to, but whenever there has been an issue or question, the reference is the ASL servers and compatibility with them.

4.2 VOTING

For voting, the Voter2 can fully inter-operate in a network with a mix of Voters and Voter2s.

4.3 SIMULCASTING

For simulcasting, the initial goal is interoperability between Voter2s only. This is in keeping with the general requirement that for simulcasting all hardware (Voters, repeaters, etc.) should be identical. As a stretch goal it would be nice to be able to mix Voters and Voter2s while still keeping the same repeaters.

4.4 TBD

5 DIFFERENCES

5.1 USER TYPES

The Voter2 supports two users at login, `root` and `guest`. Root has full access while guest cannot save changes to NVM, change passwords or do software updates. Basically, it can't make any permanent changes. The intent is letting the guest make almost any changes they like, but a reboot will bring the Voter2 back to the last root-defined configuration.

5.2 COMMAND LINE INTERFACE

The biggest, most obvious difference from the Voter is the user interface, transitioning from menu-based to command line based. Core to the Voter2 is future expandability and the menu-based interface doesn't support that gracefully. This unfortunately does mean more typing compared to the Voter. Command and parameter shortcuts hopefully help.

The `help` command lists all commands. `help` followed by a command shows a more detailed syntax for that command. Finally for commands like `set/show`, `dial`, and `local` that have sub-settings, `help` followed by the command and sub-setting will give the syntax for that sub-setting.

The command prompt starts with an optional device name (see voter name in enhancements), root/guest ID, a dirty flag, and finally a bracket. The dirty flag is set if there have been any settings changes made to memory that have not been saved to NVM. This is your reminder that some setting will be lost if you reboot.

I do plan on supporting a minimal form of command recall/editing soon. It is on the near-term to-do list.

5.3 DEBUG LOGGING

Another major difference from the Voter is both how and where debugging logs are handled.

For the Voter2 the debugging output is on a separate interface, not shared with the console. This keeps the console fully usable even if there are considerable (and ongoing) debug messages being displayed. The initial port for debugging messages is the UART on the 14-pin ST debug header (J1). This is the only port available until the RTOS and Ethernet stack are running, so any debug messaging regarding boot and the final flash reprogramming phase of a software update process are here. Once Ethernet is available you can Telnet into port 24 of the Voter2 and then debugging messages will be available there. Once the Telnet session is terminated, debugging messages revert back to the UART.

What debugging logs to show is also different. Instead of single bit-mapped value for blocks to show logging information, debugging is set by functional block name to be more expandable, then for each block there are four levels of information, `none`, `major`, `support` and `io` (most verbose). Type in `help set logging` for command details.

5.4 RSSI DETERMINATION

The Voter2 supports two sources of RSSI information. For the Voter2-2K, the primary source is intended to be the analog RSSI signal from the MTR-2000. The alternate source is similar to the Voter in that the energy level above 2.4KHz is measured and is inversely proportional to signal level. This latter method still needs to be refined!

5.5 RECEIVE LEVEL CALIBRATION

Receive audio level setting is very similar to the Voter in that you set up the 1KHz tone with 3KHz deviation on your service monitor, enter the calibration command on the Voter2 (`status level` for the Voter2), and then adjust.

The big difference for the Voter2-2K is that the analog level adjustment is done on the MTR-2000, not the Voter2! See Appendix C of the Users guide for details.

The other difference is the Voter2 has two level bars, the one on the left is the course level setting to get you close, the second is “zoomed in” on the target level to make tweaking a bit easier visually. The target is to have the level bar centered on the middle of the right hand graph.

5.6 SQUELCH

Unlike the Voter, the Voter2-2K relies on the MTR-2000s squelch control. As such, if you had been ignoring it with a Voter, you will need to calibrate it if moving to a Voter2. Since the Voter2-2K is designed specifically for the MTR-2000, I can count on the signal being available. This may not be the case for other variants of the Voter2.

5.7 SOFTWARE UPDATES

The Voter2 relies on TFTP for doing software updates. This may be a bit more cumbersome for a single stand-alone setup, but optimized for a networked environment where one central TFTP server can support all Voter2s on the network and the updates can be performed remotely (use with care!). The `set tftpaddr` command sets the address of the TFTP server. Software updates are a three-step process.

- `update check`: Queries the server for the version of software available for download and shows you the currently running version for comparison.

- `update download`: Downloads the update to a staging NVM on the voter2, but does not actually update the Voter2 yet.
- `update program`: programs the executable NVM from the staging NVM

The staging NVM step allows for a complete download and verification of the image before programming the executable NVM. This way the most vulnerable part of the update process—programming the executable flash—needs only stable power to complete, no network dependencies.

On my to do list is the ability to update via the console for non-networked setups.

5.8 SAMPLE SYNCHONIZATION

This is getting down into nuances of the protocol, but could be important for any strange artifacts with voting. Core to keeping multiple voters synchronized is the voters sending EXACTLY 8,000 samples per second—GPS-enforced using 1PPS as a reference.

For the Voter, the clock running the A/D is running from a local crystal whose frequency will be off and indeed may drift over time/temperature/etc. The Voter counts samples every GPS 1PPS and if the total number of samples is off by 1 or 2, either adds or drops a sample to get back to exactly 8,000.

For the Voter2 there is a PLL (a combo timer/software one) that generates an 8KHz sample clock that is phase and frequency locked to the GPS 1PPS, so there is never any drifting. The net result is the same, 8,000 samples per second. The difference is that for the Voter, a given second may have 7998-8002 samples, but the average will be exactly 8,000 samples. For the Voter2 it will be exactly 8,000 samples every second. They differ on instantaneous jitter. The Voter jitter is ±1-2 samples (250us) while the Voter2 is ±4 internal clocks or 400ns.

5.9 TRANSMIT DELAY

The Voter has two parameters for synchronizing audio outputs, option 7 is the main delay buffer for accommodating worst-case network delays and then option 19 for fine tuning for simulcast. The main delay is in units of a sample (125us) and fine tuning is in internal clocks (200ns).

The Voter2 (by request) combines these two parameters into a single one using real-world units, milliseconds, then does the math internally to convert it into samples and clocks. While the unit of measure is ms, this delay value can be specified down to ns using floating point notation, so up to six digits after the decimal point. There are some limitations and nuances, see the Users Guide for details.

5.10 WIRELINE CARD

The Voter2 does not require the Wireline card in the MTR2000. It talks directly to the signals on the backplane.

5.11 TBD

6 ENHANCEMENTS

6.1 REMOTE UART

The Voter2 supports the RFC2217 (COM port control over Ethernet) standard, allowing remote COM port operations. This feature along with an RFC2217 driver on your computer allows you to run the Motorola RSS software and access/control the MTR-2000 remotely. See Appendix A for details.

6.2 TEMPERATURE SENSORS

The Voter2 has two on-board temperature sensors, one internal to the MCU which is useful for ensuring the CPU isn't getting too hot (VERY UNLIKELY at 125C) and the second one external to the MCU for ambient temperature. There is support for additional sensors off-board too.

6.3 ADDITIONAL DEBUGGING FEATURES

In addition to the `set logging` command, several other debugging features exist. These have been useful to me for developing the Voter2 software, but I have deemed useful enough to retain going forward. This is a listing of the commands, see the Users guide for more details

- `netdelay` – histogram of packet delivery times to show network jitter
- `rxtest` – sends audio packets with a test pattern for sync calibration
- `txtone` – Turns on a test tone with programmable frequency and amplitude
- `set flags, set dlogger, set errlogs` – These are very "down and dirty" debugging commands only useful for development work, but if a developer is helping you with an issue, likely you will be asked to use them.

6.4 MORE (AND SEGMENTED) STATUS COMMANDS

Instead if a single status command (98), the Voter2 has a set of status commands. These commands cover everything the Voter does and more. Enter `help status` for the full list. Some new status commands to highlight are:

- `status 14v` – Compares the MTR-2000 14V power rail to 11.9V. I wanted an ADC to measure actual voltage, but ran out of ADCs, so this is all I could do. Not sure it is really useful and this feature may go away.
- `status power` – Shows the MTR-2000 AC Power status signal, noting if you are on AC power or running from batteries.
- `status rssi` – Shows the analog and high-pass filter RSSI values. Alas not available on the Voter for some reason.

- `status sync` – Pings the ASL server for a timestamp and compares it to the local time. Gives you an idea of network delays.
- `status tempr` – Shows state of the various temperature sensors installed
- `status uptime` – Shows time since last boot

6.5 BACKUP/RESTORING SETTINGS

The Voter2 has support for saving settings to a server (`settings backup`) and restoring said settings from a server (`settings restore`). This uses the same TFTP server address as used for software downloads. The settings are saved as a series of `set` commands, so you can manually copy and edit a settings file.

6.6 EXPANDABILITY

With the Voter2 nearly feature complete, only 5% of the flash is being used. There is LOTS of room for future enhancements. CPU cycles and RAM are similarly unburdened.

6.7 GPS SETTINGS

This is a minor usability upgrade. For the supported GPS modules, instead of looking up the baud rate and active edge, the `set gpsdev` command knows the settings for these GPS receivers and sets the parameters for you so you don't have to look them up. Right now, the only two supported GPS receivers are the ST Teseo LIV4F and Ublox M8N modules, but this command can be extended as more GPS receivers get support. For any one-off GPS receivers, commands for baud rate and active edge can still be set manually.

6.8 ANALOG PROBE

The advantage of doing audio processing with DSP is its flexibility. The downside is visibility, you can't normally probe the signal with your oscilloscope as it passes through the DSP chain. This command addresses that issue. It uses the second DAC port to show the signal between various filters in the Rx and Tx DSP chains. You put your scope probe on the ANLG pin of the debug header (P5) and use the `set probe` command to "probe" different points of the internal DSP chain. Of note, there is no Nyquist filter on this analog debug output.

6.9 VOTER NAME

Another usability upgrade idea from Dave (WA1JHK). You can assign a name to your Voter2 that becomes part of the command prompt using the command `set myname`. Handy when you are dealing with multiple Voter2s on the same network and can't keep track of which one you are dealing with.

6.10 TBD

7 VOTER-VOTER2 COMMAND CHEAT SHEET

This is just a quick mapping of Voter menu commands to Voter2 CLI commands with the shortcut in parenthesis. This is just the command mapping, the syntax is often different. Major differences are noted.

7.1 Main Menu

Voter	Voter2	Note
1	Show myserialnum (msn)	Set only works once! (value put in OTP)
2	Set dial fqdn (qdn)	
3	Set dial port (prt)	
4	N/A	No local port override support
5	Set dial voterpwd (vpw)	
6	Set dial hostpwd (hpw)	
7	Set dial txdly (txd)	Voter2 parameter is in decimal ms, down to ns. Combines Voter 7 and 19 commands into one.
8-9	N/A	Currently NMEA support only, no inversion support yet
10	Set gpsedge (ged)	Set gpsdev for device lookup
11	Set gpsbaud (gbd)	Set gpsdev for device lookup
12	Set rxmode (rxm)	More parameters in Voter2
13	Set rxmode (rxm)	More parameters in Voter2
14	Set logging (log)	More parameters in Voter2
15-16	N/A	No Alternate server support yet
17	N/A	N/A for Voter too
18	Set local hangtime (htm)	I think this is the equivalent?
19	Set dial txdly (txd)	Combined with command #7
97	Status level (lvl)	
98	Status xxx	Many status commands
99	Settings save	
I	See IP Table	
O	See Online Table	
S	N/A	May need something here
Q	Logout (exit)	
R	Reboot	
D	N/A	Scattered in other Voter2 commands

7.2 IP Menu

Voter	Voter2	Note
1	Set mystaticip (msi)	
2	Set mystaticnm (msn)	
3	Set mystaticgw (msg)	
4	Set mystaticdns (msd)	
5	N/A	No secondary DNS on Voter2
6	Set myipmode	
7	N/A	CLI telnet port fixed at 23
8	N/A	CLI usernames fixed at root and guest
9	Set rootpwd (rpw) Set guestpwd (gpw)	
10-13	N/A	No Dynamic DNS support
14	Set tftpaddr (tad)	Not really equivalent
15	N/A	Ethernet duplex is determined automatically
99	Settings save	

7.3 Offline Menu

Voter	Voter2	Note
1	Set local mode (mod)	
2	Set local cwspeed (cws)	
3	Set local precw (prc)	
4	Set local postcw (poc)	
5	N/A	Fixed at "OFFLINE"
6	N/A	Fixed a "ONLINE"
7	Set local id	
8	Set local hangtime (htm)	
9	Set local plfreq (plf)	
10	Set local pllevel (pll)	
11	Set local deemph (dmp)	
99	Settings save	

7.4